REHARNESS: AST-Driven Extraction of Formal Register Interaction Specifications from C Device Drivers

Anonymous Author(s)
Anonymous Institution
Anonymous City, Anonymous Country
anonymous@example.com

ABSTRACT

Device drivers account for the majority of operating system code, yet their register-level behavior remains largely implicit—buried in C source with ad-hoc macros, pointer arithmetic, and framework-dependent idioms. We present REHARNESS, a pipeline that extracts formal register interaction sequences (RIS) from C drivers using libclang AST analysis combined with flow-sensitive dataflow and taint tracking. The extracted RIS feeds backend-independent function/device specifications and deterministic userspace, bare-metal, and Linux generators. Evaluation artifacts are produced by a version-pinned, machine-readable pipeline rather than manually transcribed tables. Across 19 Linux drivers, REHARNESSextracts 469 operations: 356 symbolic, 74 fixed, and 25 computed-address accesses, including 88 read-modify-write patterns and 117 conditions. We additionally evaluate 3 real multi-source Linux modules comprising 19 translation units and 27447 lines. All three generated backends compile for every single-source case; for the multi-source modules, harness and bare-metal compile for all three while Linux compiles for DWC2 and C67X00 and conservatively rejects ASPEED vHub. Strict semantic readiness is intentionally more conservative (7/19 for Linux) because unknown transforms, unsafe dynamic addresses, and unbound subsystem state are treated as blockers rather than silently approximated. Deterministically generated drivers additionally pass value-level QEMU validation on the edu PCI device and offset/order trace validation on a real gpio-ftgpio010 driver.

KEYWORDS

device drivers, formal specification, program analysis, code generation, LLM-assisted synthesis

ACM Reference Format:
Anonymous Author(s). 2026. REHARNESS: AST-Driven Extraction of Formal Register Interaction Specifications from C Device Drivers. In . ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Device drivers are the most numerous and error-prone component of operating system kernels [1, 2]. A single Linux release contains tens of thousands of driver source files, each encoding intricate hardware protocols through memory-mapped I/O (MMIO) reads and writes, interrupt handlers, and framework callback registrations. Despite decades of research on driver reliability [3–5], the register-level behavior of drivers remains largely implicit—expressed in C source through macros, pointer arithmetic, and ad-hoc data structures that resist automated reasoning.

The core challenge is *extraction*: turning raw C source into a machine-readable specification of what the driver does to hardware registers, in what order, and under what conditions. This specification is the foundation for verification [6], testing [7], translation to safe languages [8], and AI-assisted driver synthesis [9].

Existing extraction tools, however, operate at a superficial level. They use regular expressions to match MMIO function calls (e.g., `readl`, `writel`), but fail on four common patterns: (1) register offsets defined through `#define` macros that resolve to zero under regex matching, (2) MMIO accesses hidden inside wrapper functions called from driver entry points, (3) branch conditions that govern register access paths, and (4) address expressions involving arithmetic composition of base pointers and offsets.

We present REHARNESS, a pipeline that addresses these limitations through AST-level analysis. REHARNESSuses libclang to parse C translation units with full preprocessing context, enabling it to resolve macro-expanded register offsets (e.g., `GPIO_INT_EN` → 0x20) that are invisible to regex-based tools. It performs inter-procedural call-graph analysis with controlled-depth wrapper inlining to expose MMIO operations hidden inside helper functions. It tracks branch conditions with a predicate stack that distinguishes then and negated else paths, attaching guards to each register operation and constructing conditional expressions for branch-dependent RMW updates. Critically, it performs *flow-sensitive dataflow and taint tracking*: it recognizes `ioremap` calls as MMIO base pointer sources, tracks `readl` return values as read-tainted data, and detects read-modify-write (RMW) patterns where a read value is modified and written back to the same address.

The output of REHARNESSis a formal register interaction sequence (RIS) specification language, designed to be both human-readable and machine-parseable. Each RISmodule records the sequence of register operations for a driver function, annotated with symbolic register names, branch conditions, and semantic intents.

Beyond raw extraction, REHARNESSperforms *semantic inference*: it lifts RISmodules into backend-independent function specifications (`FunctionSpec`) that capture the function's role (e.g., `interrupt_ack`), execution context, state bindings, effects, and Hoare-style pre- and postconditions. Function specs are composed into device-level specifications (`DeviceSpec`) that model the device's state, resources, register map, and invariants. These formal specs can be paired with

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

backend-specific binding files (`.bind`) to generate driver code for multiple targets: userspace harnesses for testing, bare-metal C for embedded systems, and Linux kernel modules.

Finally, REHARNESSincludes a closed-loop LLM-assisted synthesis module: when deterministic code generation is insufficient (e.g., for complex Linux subsystem glue), the pipeline assembles a structured input bundle (RIS, device spec, backend binding, source facts, scaffold) and uses compile, static, and trace verification feedback to iteratively repair LLM-generated candidates.

This paper makes the following contributions:

- A formal register interaction sequence (RIS) language that captures MMIO operations with symbolic register resolution, branch conditions, and RMW detection, serving as the canonical IR for driver behavior extraction.
- A libclang-based extraction pipeline with flow-sensitive dataflow and taint tracking that overcomes the four fundamental limitations of regex-based approaches.
- Backend-independent semantic inference from RISmodules to function- and device-level formal specifications, including callback role inference, state binding, and effect modeling.
- A multi-backend code generation framework with deterministic harness, bare-metal, and Linux backends, plus a closed-loop LLM-assisted synthesis module with structured verification feedback.
- A reproducible evaluation on 19 Linux drivers, with generated tables tied to machine-readable results, 19/19 compilation for all three backends, and deterministic QEMU validation on PCI and platform devices.

2 MOTIVATION

2.1 The Limits of Regex-Based Extraction

To motivate REHARNESS, we examine the failure modes of the state-of-the-practice regex-based extraction tool, `driver-harness` [10]. Its approach scans C source text with regular expressions to detect MMIO calls. While lightweight, this strategy fails systematically on four patterns pervasive in real driver code.

Pattern 1: Macro-defined register offsets. Linux drivers define register layouts through preprocessor macros:

```
#define GPIO_INT_EN   0x20
#define GPIO_INT_CLR  0x30
writel(0, base + GPIO_INT_EN);
```

A regex that matches `writel(..., ... + ...)` can capture the call site but cannot resolve `GPIO_INT_EN` to its numeric value 0x20. The baseline tool defaults to reporting offset 0 for all such accesses, losing the essential information that distinguishes one register from another. In our evaluation, this causes 100% of register accesses in GPIO and virtio-MMIO drivers to be incorrectly reported as residing at offset 0.

Pattern 2: Wrapper functions. Drivers commonly wrap MMIO primitives in helper functions:

```
static void gpio_setbit(u32 val, void __iomem *
    base, u32 off) {
    u32 r = readl(base + off);
    writel(r | val, base + off);
}
```

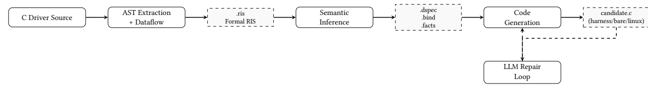

Figure 1: REHARNESSpipeline. C source is parsed via libclang to extract formal RIS; semantic inference enriches RIS into backend-independent specs; code generation targets multiple backends; a dashed LLM repair loop patches candidates under verification feedback.

A regex operating on the primary callback function will miss all MMIO operations inside `gpio_setbit` and similar wrappers. Without inter-procedural analysis, the entire register access footprint of the driver is incomplete.

Pattern 3: Conditional register access. Register sequences often depend on runtime conditions:

```
if (type & IRQ_TYPE_EDGE_BOTH) {
    writel(val | BIT(line), base +
        GPIO_INT_BOTH_EDGE);
}
```

The baseline tool records the `writel` operation but discards the guard condition, losing the semantic context that the write only occurs for edge-triggered interrupts.

Pattern 4: Arithmetic address computation. Base pointers are often computed through field access chains:

```
struct ftgpio_gpio *g = gpiochip_get_data(gc);
readl(g->base + GPIO_INT_STAT);
```

Resolving `g->base` requires tracking the `ioremap` call that initialized it, which in turn requires flow-sensitive taint propagation—well beyond the reach of regular expressions.

2.2 Design Goals

REHARNESSis designed to overcome these limitations while preserving three properties essential for downstream toolchains:

1. **Precision:** Every register access must be retained in its most precise sound address class: symbolic name and offset when statically derivable, otherwise an explicit fixed or runtime-computed address rather than a fabricated symbol.
2. **Completeness:** No MMIO operation reachable from a driver entry point should be silently dropped. Wrapper inlining and inter-procedural analysis must cover the full call graph to a bounded depth.
3. **Formal Semantics:** The output must be a formal specification language, not an ad-hoc JSON format, enabling direct use as input to verification, testing, and code generation tools.

3 SYSTEM OVERVIEW

REHARNESSis organized as a pipeline with four major stages, illustrated in Figure 1.

Stage 1: RIS Extraction. The extraction stage parses C source with libclang, builds a macro offset table from preprocessing records, identifies driver entry points, and performs per-function flow-sensitive dataflow analysis. The output is a formal RISspecification recording every register read, write, and RMW operation with symbolic register names, branch guards, and width annotations.

Stage 2: Semantic Inference. The inference stage enriches raw RISmodules with backend-independent semantics. It infers function roles from callback table registrations (e.g., `.irq_ack = ftgpio_ack_irq` → `interrupt_ack`), abstracts C parameter types to formal types (e.g., `struct irq_data *` → `LogicalIRQ`), binds MMIO bases to device state, and derives effects and Hoare-style contracts from role semantics. The result is a `FunctionSpec` per driver function, composed into a `DeviceSpec` with state, resources, register map, and invariants.

Stage 3: Code Generation. The generation stage consumes the formal specs and backend-specific binding files to produce driver code for three targets: (1) a userspace harness with fake MMIO memory and trace logging for testing, (2) portable bare-metal C for embedded systems, and (3) Linux kernel modules with platform, PCI, GPIO/IRQ, clock-provider, and miscdevice lifecycle glue selected from the inferred device class. Conservatively recognized source-private clock containers are rebound to generated state while retaining their scalar rate arithmetic, pure helpers, and distinct callback-table instances. When an operation cannot be bound explicitly, generation emits an unsupported marker and neutral code instead of inventing behavior.

Stage 4: LLM-Assisted Synthesis. When the deterministic backend cannot fully reconstruct source-private state or complex subsystem semantics, REHARNESSassembles a structured bundle of RIS, device spec, backend binding, source facts, and a deterministic candidate. It then enters a closed loop: compile/static/trace verification yields structured feedback, which is fed to an LLM as a repair prompt. The LLM patches the candidate, and the loop repeats until the candidate passes all checks or the iteration budget is exhausted.

4 FORMAL RIS LANGUAGE

The core output of REHARNESSis the RIS(Register Interaction Sequence) specification language. RISis designed to be both human-readable and machine-parseable, with a formal grammar supporting the full vocabulary of MMIO operations found in real drivers.

4.1 Grammar

```
driver <name> v<version> {
  module <function_name> {
    <var> := R(<width>, <addr>) [-- <intent>]
    W(<width>, <addr>) = <expr> [-- <intent>]
    RMW(<width>, <addr>) = <transform> [-- <intent
        >]
    IF <guard> { <ops> } [ELSE { <ops> }]
    LOOP <count> { <ops> }
    DELAY(<cycles>)
  }
}
```

Each `module` corresponds to a driver function (a framework callback or helper). Operations within a module are ordered as they appear in source, preserving the program's execution sequence.

4.2 Operations

Read (`R`): A register read with a bound variable for later use. Width is one of `B1`, `B2`, `B4`, `B8` (inferred from the MMIO call's suffix: `readl` → B4, `readw` → B2, etc.). The address is resolved to one of three forms:

- **Symbolic:** `g->base.GPIO_INT_EN`—a known base expression plus a resolved register macro.
- **Fixed:** `0x20`—a literal offset with no macro resolution.
- **Computed:** `[g->base + idx]`—a dynamic address expression that could not be statically resolved.

Write (`W`): A register write with an expression value, which may be a constant, a variable, or a compound expression in the `Expr` algebra.

Read-Modify-Write (`RMW`): Detected when a `writel` value is the result of modifying a `readl` return from the same address. This is a critical pattern for interrupt mask/unmask operations, where a single bit must be toggled without disturbing others.

Conditional (`IF`): Branch conditions extracted from `if/else` statements and attached to operations within each branch. A predicate stack retains nested guards and distinguishes then paths from negated else paths.

Loop (`LOOP`): Iteration constructs from `for/while` loops, annotated with a count expression when statically derivable.

Delay (`DELAY`): Timing operations (`mdelay`, `udelay`, `msleep`) converted to nanosecond-equivalent counts.

4.3 Expression Algebra

Values and guards in RISare represented in an algebraic expression language:

$$\texttt{Expr} ::= \texttt{Const}(n) \mid \texttt{Var}(x) \mid \texttt{BinOp\{op, left, right\}} \mid \texttt{Ite\{guard, then, else\}} \mid \texttt{Bits\{hi, lo, expr\}} \mid \texttt{Top}$$

`BinOp` supports arithmetic, bitwise, relational, and logical operators. `Ite` represents a path-conditioned value and is emitted as a C conditional expression by all backends. The `Top` sentinel represents values that could not be statically resolved.

Example expressions:

```
0x1 << irqd_to_hwirq(d)
  -> BinOp{Shl, Const(1), Var("irqd_to_hwirq(d)")}
0x0 ^ 0xffffffff
  -> BinOp{BitXor, Const(0), Const(0xffffffff)}
```

4.4 Example

Listing 1 shows the extracted RISfor the `gpio-ftgpio010` driver, a Faraday FTGPIO010 GPIO controller. Each module corresponds to a callback registered through the Linux `irq_chip` and `gpio_chip` framework tables.

Listing 1: Excerpt from the extracted RISspecification for gpio-ftgpio010 (14 modules, 32 ops including generic-library summaries)

```
driver gpio-ftgpio010 v0.1.0 {
  module ftgpio_gpio_ack_irq {
    W(B4, g->base.GPIO_INT_CLR) = (0x1 <<
        irqd_to_hwirq(d))
         -- Interrupt
  }
  module ftgpio_gpio_mask_irq {
    val := R(B4, g->base.GPIO_INT_EN) -- Interrupt
    RMW(B4, g->base.GPIO_INT_EN) = val --
        Interrupt
  }
  module ftgpio_gpio_unmask_irq {
    val := R(B4, g->base.GPIO_INT_EN) -- Interrupt
    RMW(B4, g->base.GPIO_INT_EN) = val --
        Interrupt
  }
  module ftgpio_gpio_set_irq_type {
    reg_type  := R(B4, g->base.GPIO_INT_TYPE) --
        Interrupt
    reg_level := R(B4, g->base.GPIO_INT_LEVEL) --
        Interrupt
    reg_both  := R(B4, g->base.GPIO_INT_BOTH_EDGE)
         -- Interrupt
    RMW(B4, g->base.GPIO_INT_TYPE) = reg_type
    RMW(B4, g->base.GPIO_INT_LEVEL) = reg_level
    RMW(B4, g->base.GPIO_INT_BOTH_EDGE) = reg_both
  }
  module ftgpio_gpio_irq_handler {
    stat := R(B4, g->base.GPIO_INT_STAT_RAW) --
        Interrupt
  }
  module ftgpio_gpio_set_config {
    val := R(B4, g->base.GPIO_DEBOUNCE_PRESCALE)
        -- Config
    IF (val == deb_div) {
      val := R(B4, g->base.GPIO_DEBOUNCE_EN) --
          Config
      RMW(B4, g->base.GPIO_DEBOUNCE_EN) = val --
          Config
    }
    ...
  }
  module ftgpio_gpio_probe {
    W(B4, g->base.GPIO_INT_EN)   = 0x0 -- Init
    W(B4, g->base.GPIO_INT_MASK) = 0x0 -- Init
    ...
  }
}
```

All 22 direct source register operations plus 10 operations recovered from the typed generic-GPIO configuration are resolved to symbolic addresses (100% symbolic rate). The mask/unmask callbacks correctly exhibit RMW patterns (read the current interrupt enable mask, modify it, write it back). The `set_config` module preserves the conditional guard (`val == deb_div`) around the debounce-enable RMW.

5 RIS EXTRACTION VIA AST ANALYSIS

5.1 Translation Unit Parsing

REHARNESSuses libclang to parse each C source file as an unsaved translation unit, enabling preservation of macro expansion records. The parser operates in tolerant mode, continuing past syntax errors that would halt a regular compiler (essential for drivers that depend on kernel headers not available in the analysis environment). Parsing yields a complete AST cursor tree plus a `MacroTable` built from preprocessing records.

The `MacroTable` maps every `#define` in the translation unit to its expanded value. For integer-valued register macros (e.g., `#define GPIO_INT_EN 0x20`), we record the numeric offset. This table is the critical data structure that enables symbolic address resolution: when the dataflow analyzer encounters a reference to `GPIO_INT_EN` in an address expression, it can resolve it to 0x20 and pair it with the base pointer to produce a `Symbolic` address.

5.2 Function Identification and Callback Detection

Target functions are identified by traversing the AST for function definitions whose source location is in the target file. Each function is modeled as a `Func` object with its name, parameter list, return type, cursor, source path, and linkage identity. Externally visible definitions retain their linker symbol, whereas file-local functions use a source-qualified identity. Consequently, two translation units may both define `static helper` without either rejecting the module or propagating one helper's MMIO summary into the other.

Callback entries are recovered from designated initializers and member assignments, while tracking the enclosing structure type. The field and structure are matched against a role table (`FIELD_ROLE`); for example, `.irq_ack = ftgpio_gpio_ack_irq` becomes the full binding `irq_chip.irq_ack` with role `interrupt_ack`. Functions without a recognized framework binding are classified as helpers and may be inlined into callers.

5.3 Flow-Sensitive Dataflow and Taint Tracking

The core of the extraction is a per-function flow-sensitive dataflow analysis. For each function, we walk its call sites in source order, maintaining an abstract store mapping variable names to abstract values (`AbsVal`) drawn from a finite domain:

$$\texttt{AbsVal} ::= \texttt{BasePtr}(b) \mid \texttt{Offset}(b,\ n,\ r) \mid \texttt{ReadTaint}(a,\ r) \mid \texttt{Const}(n) \mid \texttt{SymExpr}(e) \mid \texttt{Top}$$

Taint sources: `ioremap` and its direct or devres-managed variants are recognized as MMIO base-pointer sources. The variable receiving the mapping is bound to `BasePtr`.

Address resolution: When an MMIO read/write call is encountered, its address argument is symbolically evaluated against the abstract store and macro table. The evaluation resolves `base + REG` expressions by combining the `BasePtr` for the pointer variable with the `Offset` value from the macro table, producing a `Symbolic` address carrying both the device context and the register name.

Read taint: The return value of a `readl` call is bound to `ReadTaint(addr)`, recording which address was read. When a subsequent `writel` uses a value that evaluates to a ReadTaint of the *same* address, REHARNESSdetects a read-modify-write pattern and emits an RMW operation instead of separate read and write.

Expression evaluation: A recursive descent evaluator handles arithmetic (+, -, «, »), bitwise (|, &, ^), and unary (~) operations over abstract values. Constant folding is applied where both operands are Const; otherwise, the expression is captured as a SymExpr or BinOp for the formal RISoutput.

5.4 Wrapper Function Inlining

When a call site references an analyzed function that is not a framework primitive (not in FRAMEWORK_FNS), REHARNESSchecks whether that function has been previously extracted. If so, its operations are instantiated at the call site: formal parameters are replaced by actual argument expressions in addresses, values, variables, and branch guards, and the call-site predicate is applied to every propagated operation. The same mechanism operates across translation units using linker/source-qualified symbol identities. Summary expansion is iterative and depth-bounded (default 3) to prevent infinite recursion through mutually recursive helpers.

This is a lightweight form of inter-procedural analysis: we do not perform full context-sensitive summary computation, but the bounded-depth inlining is sufficient to expose MMIO operations hidden in common wrapper patterns such as gpio_setbit, reg_read, and reg_write.

5.5 Intent Annotation

Each operation is annotated with an *intent* string derived from the resolved register macro name. We classify register names using keyword matching: registers containing "INT" or "IRQ" are assigned intent Interrupt; "CLK" or "PLL" yield Clock; "CONFIG" or "CTRL" yield Config; "STAT" yields Status; and so on. This lightweight heuristic provides human-readable context without requiring deep semantic analysis.

6 SEMANTIC INFERENCE

The raw RIScaptures what registers are accessed and how, but does not explain *why*—what role each function plays in the driver, what effects it has on device state, and what contracts it must satisfy. REHARNESSenriches RISmodules with backend-independent semantics through three layers of inference.

6.1 Function Specification (FunctionSpec)

A FunctionSpec is a tuple:

$$(\text{Signature}, \text{Role}, \text{Context}, \text{Binds}, \text{Requires}, \text{Ensures}, \text{Effects}, \text{RISRef})$$

Role inference is the critical step. REHARNESSprioritizes callback table bindings over function-name heuristics. When a callback table field is recognized (e.g., .irq_ack in an irq_chip structure), the corresponding role (interrupt_ack) and execution context (irq) are assigned deterministically. The table covers standard Linux fields across irq_chip, platform_driver, virtio_config_ops, gpio_chip, dev_pm_ops, usb_ep_ops, usb_gadget_ops, and hc_driver.

For functions not appearing in any callback table, a fallback heuristic matches function name substrings to roles (e.g., *_ack_irq → interrupt_ack).

Signature abstraction maps C parameter types to abstract formal types: struct irq_data * → LogicalIRQ, struct platform_device * → DeviceState, integer types → UInt, void → Void. This abstraction is essential for backend independence: the same DeviceSpec can target Linux, bare-metal, and harness backends without referencing Linux-specific types.

State binding identifies the MMIO base expression used by the function (e.g., g->base) and binds it to a formal dev.base path. The binding is discovered by scanning the address expressions of the function's RISoperations for member-access patterns.

Effects are derived from two sources: (1) for each symbolic register written by the function, a writes_register(REG) effect is emitted; (2) from the function's role, a semantic event effect is derived (e.g., interrupt_ack → clears_interrupt(line)).

Requires/Ensures are Hoare-style pre- and postconditions derived from role semantics. For example, an interrupt_ack function requires nothing and ensures interrupt_pending[line] == false; a probe function requires resources_available and ensures device_state == READY.

6.2 Device Specification (DeviceSpec)

A DeviceSpec composes multiple FunctionSpec instances into a device-level model:

$$(\text{State}, \text{Resources}, \text{Registers}, \text{Functions}, \text{Invariants}, \text{Class})$$

Device class is inferred from the driver name (e.g., gpio-ftgpio010 → gpio_controller, virtio_mmio → virtio_mmio).

State is inferred from the functions' needs: every device gets an MmioBase state field; a consumer-side Clock field is added only when the source declares or acquires a struct clk (a clock provider with clk_hw/clk_ops does not create a false consumer resource); and interrupt-related roles add num_irqs: UInt.

Resources are derived from the state fields: an MmioResource bound to base, plus ClockResource and IrqResource as needed.

Registers are collected from the RIS's register_map, which contains every symbolic register actually accessed by the driver (not every register the hardware defines, only the ones the driver touches).

Invariants are generated as minimal safety properties: for interrupt-capable devices, the invariant forall line: UInt. line < num_irqs -> valid_interrupt_line(line) is asserted.

6.3 Backend Binding (.bind)

A BindSpec maps abstract concepts from the DeviceSpec to concrete backend APIs:

- **Type maps:** DeviceState → "struct ftgpio_gpio", MmioBase → "void __iomem *" (Linux) or uintptr_t (bare-metal).
- **Callback maps:** irq_chip.irq_ack = ftgpio_gpio_ack_irq.
- **Primitive maps:** MmioRead(B4) → "readl" (Linux) or mmio_read32 (bare-metal).
- **State maps:** dev.base → "g->base".

- **Export maps:** interrupt_ack as "ftgpio_ack_irq" (bare-metal).

Default bindings are generated per backend, but can be customized to support non-standard APIs or naming conventions.

6.4 Source Facts (.facts)

For LLM-assisted synthesis, REHARNESSextracts *source facts*—information that aids reconstruction but should not pollute the backend-independent formal spec:

- **Includes:** <linux/gpio/driver.h>, <linux/platform_device.h>.
- **Struct definitions:** ftgpio_gpio with fields base: void __iomem *, clk: struct clk *.
- **Constants:** Non-register macros with integer values (flags, status codes, bit masks).
- **Callback tables:** irq_chip.irq_ack = ftgpio_gpio_ack_irq, etc.
- **Resources:** Acquisition calls (devm_platform_ioremap_resource) with bind targets.
- **Error paths:** Return codes found in the source (-ENOMEM, -ENODEV, PTR_ERR).
- **Helper calls:** Notable subsystem functions (devm_gpiochip_add_data, bgpio_init).

7 CODE GENERATION

REHARNESSincludes a multi-backend code generator that consumes (RIS, DeviceSpec, BindSpec) to produce compilable driver code. Three deterministic backends are provided; a fourth backend uses LLM-assisted synthesis for complex targets.

7.1 Userspace Harness Backend

The harness backend generates a self-contained C program that emulates MMIO through a fixed-size memory buffer (uint32_t mmio_mem[4096]). Reads and writes are routed through wrapper functions (harness_read32, harness_write32) that log every access with a trace line:

```
[trace 0] W 0x20 = 0x00000000  ; GPIO_INT_EN
[trace 1] W 0x30 = 0xffffffff  ; GPIO_INT_CLR
```

The harness includes the device state struct, register constants derived from the register map, RIS-backed function bodies (using the backend binding's primitive and state maps), and unit-test scaffolding. The entire harness compiles with a standard C compiler (cc -Wall) and produces a binary that can be executed and traced.

7.2 Bare-Metal Backend

The bare-metal backend generates portable C targeting freestanding environments. Instead of Linux kernel types, it uses standard integer types (uintptr_t, uint32_t) and portable MMIO primitives (mmio_read32, mmio_write32). The generated code includes the device state struct, init/reset/IRQ helper functions, and no framework glue. It compiles with cc -ffreestanding -c.

7.3 Linux Backend

The Linux backend generates buildable kernel modules with struct device_state, probe/remove functions, framework ops tables, and RIS-backed callback bodies using proper Linux MMIO primitives (readl, writel). It provides deterministic resource lifecycles for platform MMIO devices (devres mapping, optional consumer clocks, GPIO/IRQ registration, clock-provider registration, and OF matching), PCI MMIO devices (enable/request/map/unmap/release and PCI IDs), and QEMU edu (miscdevice open/read/write operations that expose the MMIO oracle). For conservatively recognized clock providers, distinct clk_ops instances and non-MMIO rate arithmetic are retained from the versioned source after explicit private-container rebinding. A similarly conservative source model recovers the native-endian 32-bit dat/set/dirout GPIO pattern and generic-chip mask/unmask/EOI contract used by Sodaville; unsupported variants are rejected rather than approximated. Unsupported source-private state is neutralized only to keep the result buildable and is accompanied by a REHARNESS_UNSUPPORTED marker that prevents strict-readiness claims.

7.4 LLM-Assisted Synthesis Loop

When deterministic generation is insufficient—for example, when the Linux backend cannot fully reconstruct the probe/resource lifecycle or subsystem-specific registration glue—REHARNESSenters a closed-loop LLM-assisted synthesis mode (Figure 1, dashed path).

The synthesis module assembles a structured input bundle: device.ris, device.dspec, device.<backend>.bind, device.facts, device.scaffold.c, and two guiding documents (constraints.md and verification.md). The scaffold is the output of the deterministic generator.

The repair loop operates as follows:

1. **Compile:** The candidate is compiled with strict warnings for the target backend.
2. **Static checks:** Every RISreference resolves to a module; every symbolic register is in the register map; every callback entry has a table binding; no TODOs remain.
3. **Trace check** (harness only): The generated harness is executed and its MMIO trace is compared to the expected trace derived from the RISspecification.
4. **Repair:** If any check fails, structured feedback is formatted and sent to an LLM along with the current candidate. The LLM is instructed to produce a patched C candidate that satisfies all constraints.
5. **Repeat:** Steps 1–4 iterate until the candidate is accepted or the iteration budget (default 3) is exhausted.

The LLM is a pluggable component implemented through the Pi agent SDK; it is not required by the deterministic evaluation. The reproducible matrix and the two QEMU experiments reported below invoke only the AST extractor and deterministic generators. LLM-assisted runs are evaluated separately because model and endpoint nondeterminism would otherwise confound artifact reproduction.

8 EVALUATION

We evaluate REHARNESSon 19 versioned Linux driver sources covering GPIO, clock/PLL, AHCI, SDHCI, virtio-MMIO, a framebuffer raster engine, and QEMU edu. The evaluation addresses five research questions:

- **RQ1 (Precision):** What fraction of register accesses are resolved to symbolic addresses?

Table 1: Extraction statistics across 19 test drivers. Sym, Fixed, and Comp are distinct address classes.

Driver	Ops	Sym	Fixed	Comp	RMW	Cond	Regs	%Sym
gpio-ftgpio010	32	32	0	0	10	2	13	100.0%
virtio_mmio	60	44	15	1	0	44	31	73.3%
gpio-pl061	31	27	0	4	7	1	7	87.1%
gpio-cadence	30	30	0	0	8	8	12	100.0%
edu	5	3	2	0	0	2	3	60.0%
Total (19)	**469**	**356**	**74**	**25**	**88**	**117**	**157**	**78.2%**

- **RQ2 (Completeness):** How many RMW patterns and branch conditions are detected?
- **RQ3 (Generation):** Do the generated harness, bare-metal, and Linux targets compile?
- **RQ4 (Comparison):** How does REHARNESScompare to the regex-based baseline?
- **RQ5 (Runtime):** Do deterministically generated drivers preserve register behavior on real QEMU devices?

8.1 Experimental Setup

All experiments were run on a Linux machine with an AMD 9950X processor and 64 GB RAM, using Python 3, libclang 18, and the pinned Linux submodule commit recorded in `experiments/results/matrix.json`. The kernel configuration (including `CONFIG_COMMON_CLK=y`), QEMU scripts, driver corpus, and source hashes are versioned with the artifact. The complete 19-driver extraction/generation/Kbuild matrix takes approximately 108.0 seconds (5.69 seconds per driver on average) after the experiment kernel has been built.

8.2 Extraction Coverage (RQ1, RQ2)

Table 1 is generated directly from the machine-readable experiment result and summarizes extraction across all 19 drivers.

Key finding: REHARNESSpreserves three address classes instead of forcing all accesses into a symbolic-name category. Of the address-bearing operations, 78.2% are symbolic, while fixed offsets and runtime-computed addresses remain explicit. This corrects an earlier evaluation mistake that conflated "resolved" with "symbolic." The distinction matters: indexed virtio configuration accesses and banked GPIO registers are representable but cannot honestly be assigned one static register name.

The 88 detected RMW patterns are concentrated in interrupt mask/unmask callbacks and configuration functions. Straight-line mutations such as `val &= ~BIT(line)` are retained as transforms. Branch-dependent switch updates are represented as nested `Ite` expressions, with every branch applied independently to the original read value rather than incorrectly concatenated.

The 117 recorded branch conditions occur primarily in configuration and probe functions that check device state before modifying registers.

8.3 Multi-Source Driver Scaling

The initial corpus treats one C translation unit as one extraction target. To evaluate realistic module scale, we added versioned manifests for Kbuild modules composed from four or more C files. Each manifest is checked against the pinned Linux module's actual `*-y` object list; the files are not unrelated drivers grouped after the fact.

Table 2: Real multi-source Linux driver modules. X-TU is the number of resolved cross-translation-unit call edges and Prop is the number of call edges carrying MMIO summaries. H/B/L reports generated-backend compilation. † denotes a successful original Kbuild compile/link after warning-only modpost retry because the fixed experiment kernel omits USB core exports.

Driver module	TUs	LoC	Funcs	X-TU	Prop	Src MMIO	RIS MMIO	Ops	H/B/L	Orig. Kbuild
aspeed-vhub	5	3540	92	53	54	101	150	154	✓/✓/–	✓†
c67x00	4	2239	89	30	79	2	32	38	✓/✓/✓	✓†
dwc2	10	21668	445	140	445	804	3560	4202	✓/✓/✓	✓†

Table 3: Generation readiness scores per driver.

Driver	RIS	FuncSpec	DevSpec	H/B/L compile	Linux ready	LLM ready
gpio-ftgpio010	1.000	1.000	1.000	✓/✓/✓	✓	✓
virtio_mmio	0.888	0.812	1.000	✓/✓/✓	–	✓
gpio-pl061	0.924	1.000	0.938	✓/✓/✓	✓	✓
gpio-cadence	1.000	1.000	1.000	✓/✓/✓	✓	✓
edu	0.920	1.000	0.688	✓/✓/✓	✓	✓

The extractor builds every translation unit with its own preprocessing context, merges macros and source facts, and iteratively propagates parameter-instantiated MMIO summaries across file boundaries before constructing one combined RIS and DeviceSpec. The experiment additionally compiles the original module from a temporary copy of its source directory and compares lexical source MMIO primitives with direct and propagated RIS operations.

Table 2 covers 3 modules, 19 translation units, 27447 source lines, and 626 functions, yielding 4394 operations, 948 RMW patterns, and 68 mapped registers. Of 974 internal call edges, all 223 cross-TU edges are resolved; 578 edges carry MMIO summaries. The source comparison finds 907 lexical MMIO primitives and 3742 emitted RIS MMIO operations after propagation. C67X00 demonstrates propagation through its low-level HPI implementation, ASPEED vHub covers endpoint/hub paths, and the ten-file DWC2 case exercises host, gadget, dual-role, endpoint, and DMA code. Harness and bare-metal compile for all three modules; generated Linux compiles for DWC2 and C67X00. C67X00 now binds HPI base, register stride, and SIE number as source-private state and lowers all 32 computed HPI addresses. ASPEED vHub retains explicit endpoint-lifecycle and subsystem-state blockers. Compilation remains distinct from strict readiness: C67X00 still records a switch-exclusivity gap, an unproved loop, and unsupported external HCD lifecycle semantics.

8.4 Generation Readiness (RQ3)

Table 3 separates compilation from strict semantic readiness. A generated file can compile while still carrying an explicit unsupported state-binding marker.

Harness, bare-metal, and Linux outputs compile for 19/19 drivers. In contrast, strict readiness is 2/19 for harness, 2/19 for bare-metal, and 7/19 for Linux; 13/19 have sufficient structured context for assisted synthesis. Generic GPIO callbacks recovered from `gpio_generic_chip_config` are compiled and lowered by the Linux backend, but generic harness/bare-metal readiness remains blocked until those synthesized callbacks have an execution oracle; this

Table 4: Comparison of REHARNESSvs. regex baseline on four key patterns.

Pattern	Regex Baseline	REHARNESS
Macro register offsets	0% resolved	Constants recovered from AST
Wrapper function inlining	MMIO lost	Inlined (depth ≤ 3)
Branch conditions	Always None	117 conditions recorded
Arithmetic addresses	Not supported	Symbolic/fixed/computed preserved
RMW detection	Not supported	88 RMW patterns

removes four earlier false-ready cases. Highbank separately illustrates source-aware Linux-only clock lowering. Unknown transforms, unsafe computed addresses, target-source clang errors, unsupported subsystem domains, and normalized source-private state block readiness even when code can be compiled. The mean RIS quality is 0.907.

We additionally publish `experiments/results/reliability.json`. Each recognized source access receives a stable site identifier, every RIS leaf carries source evidence and precision labels, and path predicates are checked for satisfiability and switch exclusivity with Z3. Direct volatile dereferences, inline assembly, unsupported register domains, and unmodeled control transfers cannot disappear silently. The report explicitly scopes its strict verdict to recognized register-access and control patterns. `whole_program_complete` is now a conjunction over linked analysis, all translation units, internal calls, access accounting, explicit CFG, path validation, switch exclusivity, operation identity/evidence, computed addresses, values, loops, and access domains. Its scope is `manifest-internal`; it does not claim completeness for external kernel or subsystem semantics. The default single-source evaluation leaves alias analysis off and therefore remains false.

We separately record a machine-readable clock-model boundary in `experiments/results/clock-model-boundary.json`. Generated Highbank callbacks pass 22 arithmetic cases against an independent Python reference, while mutations to PLL divq, A9 bus shift, and peripheral-clock increment formulas are all detected. Applying the same source model to Visconti PLL is rejected with explicit unbound-state reasons (PLL base, rate table/count, and lock). This negative control demonstrates a conservative syntax/ownership boundary; the Highbank oracle validates the versioned formulas but is not a hardware-equivalence proof.

For C67X00, a required-mode manifest run compiles each translation unit to bitcode, links all four units, and invokes one SVF WPA pass. The analysis records a reproducible linked-bitcode SHA-256 and source/tool provenance. The linked run finds no additional ordinary pointer aliases, supporting the diagnosis that the hard part is HPI aggregate state rather than alias discovery. A separate oracle executes five source primitives and four original-C-versus-RIS differential cases; mutations to register index, register stride, wrapper target, and operation order are all detected.

8.5 Comparison to Regex Baseline (RQ4)

We compare REHARNESSagainst `driver-harness`, a regex-based tool that served as REHARNESS's predecessor and baseline. The comparison focuses on the four failure patterns identified in Section 2.

The regex baseline resolves 0% of macro-defined register offsets because it cannot evaluate preprocessor constants. It misses MMIO operations inside wrapper functions, reports `None` for branch conditions, and cannot classify arithmetic address expressions. REHARNESSaddresses these patterns through AST-level analysis while preserving the distinction between 356 symbolic, 74 fixed, and 25 computed-address operations. It therefore avoids both the baseline's lost accesses and the unsound claim that every representable address has one static symbolic name.

8.6 Case Study: gpio-ftgpio010

We examine the FTGPIO010 GPIO controller driver in detail. The target file directly defines 7 MMIO-bearing modules; the generic-GPIO library contract contributes 7 additional synthesized callbacks from the typed configuration. The driver defines 13 register macros, all of which are now referenced by direct or library-summary operations.

REHARNESSextracts 32 operations across 14 modules: 22 direct operations and 10 typed library-summary operations. It resolves 13 registers, detects 10 RMW patterns, and records 2 conditions. Direct callbacks retain their enclosing-structure bindings, while synthesized callbacks are bound to `gpio_chip.get/set/get_multiple/set_multiple/` fields. The three switch-dependent RMW operations in `set_irq_type` remain nested `Ite` transforms with no `Top`.

The generated userspace harness compiles with `cc -Wall`, the bare-metal C compiles with `cc -ffreestanding -c`, and the Linux output builds through the pinned kernel's Kbuild interface. In QEMU, a versioned platform-device registrar triggers the generated driver's probe path and a GPIO chardev exerciser invokes generic callbacks. Its instrumented trace matches the RIS-derived oracle with 6/6 module, 7/7 call, 16/16 operation, and 7/7 register coverage (`TRACE_MATCH_OK`).

8.7 Deterministic Runtime Validation (RQ5)

The runtime experiments are executed by `verification/run_qemu_experiments.sh` and recorded in `experiments/results/qemu.json`; neither experiment invokes an LLM. For QEMU edu, the deterministic PCI backend generates a miscdevice that provides positional reads and writes over the mapped BAR. The guest exerciser validates the device ID at offset 0x00, the complemented live-check value at 0x04, and the factorial result at 0x08. All value-level checks pass (`EDU_TRACE_OK`).

For gpio-ftgpio010, the experiment registers a platform device, loads the instrumented deterministic module, and runs a GPIO chardev exerciser that triggers generic-chip direction, get-multiple, and set-multiple callbacks. Instrumentation emits exact runtime-function boundaries, and the oracle matches each selected call segment once against structured Formal RIS operations, preventing one global trace from being credited to multiple callbacks. The oracle passes (`TRACE_MATCH_OK`) with 6/6 module coverage, 7/7 call coverage, 16/16 operation coverage, and 7/7 register coverage. These experiments validate two complementary properties: concrete read/write values on a modeled PCI device and access order/offset preservation across a Linux platform-driver probe and exercised subsystem callbacks.

9 RELATED WORK

9.1 Driver Analysis and Extraction

C-to-Rust translation. The C2Rust project [8] performs syntactic translation from C to unsafe Rust, leaving the insertion of safe ownership and borrowing annotations to subsequent tools. &inator [17] addresses the interface translation problem by modeling Rust type selection as an SMT constraint satisfaction problem over type fragments, pointer transformation graphs, and NLL borrowing constraints. REHARNESSis complementary: it extracts formal register-level behavior that can guide the semantic preservation constraints in C2Rust-style translators.

Static driver analysis. Tools such as SDV [3], Dingo [4], and Termite [12] perform static analysis on driver source to verify protocol compliance and generate boilerplate. These tools operate at the framework API level (e.g., verifying that `probe` allocates resources and `remove` frees them). REHARNESSoperates at a lower level: it extracts the actual register interaction sequences, which capture the hardware protocol rather than the OS framework protocol.

Symbolic execution. KLEE [7] and S2E [14] perform symbolic execution of driver code to generate test inputs. REHARNESS's extraction is static rather than dynamic, making it applicable to drivers that cannot be executed in a test environment (e.g., due to missing hardware or kernel infrastructure).

9.2 Formal Specification and Code Generation

Device driver synthesis. Termite-2 [13] synthesizes drivers from formal specifications of device and OS interfaces. The specifications must be written manually in a domain-specific language. REHARNESSautomates the specification extraction step, producing `DeviceSpec` and `BindSpec` automatically from existing C source.

LLM-based code generation. Recent work explores using large language models for driver synthesis [9]. These approaches typically operate on natural language descriptions or source snippets. REHARNESS's LLM-assisted synthesis module is unique in providing structured formal specifications and verification feedback as input constraints, enabling a repair loop rather than one-shot generation.

9.3 Intermediate Representations for Drivers

Devicetree and ACPI. Hardware description standards such as Devicetree and ACPI describe the hardware topology (register base addresses, interrupt lines, clocks) but not the register interaction protocols. REHARNESS's RISand `DeviceSpec` complement these standards by capturing the behavioral protocol.

Haiku and Barrelfish driver models. The Haiku [16] and Barrelfish [15] operating systems use driver models that separate device logic from OS framework code, similar to REHARNESS's separation of `DeviceSpec` (what the device does) from `BindSpec` (how it connects to an OS). REHARNESSprovides the tooling to extract these specifications from existing Linux drivers.

10 DISCUSSION AND LIMITATIONS

10.1 Current Limitations

Linux subsystem coverage. REHARNESSrecognizes GPIO and IRQ tables (`gpio_chip`, `irq_chip`, and `gpio_irq_chip`); bus drivers (`platform_driver`, `pci_driver`, and `amba_driver`); virtio, clock, power-management, file-operation, and USB endpoint/gadget/HCD tables. USB callback signatures and common private fields are modeled, but endpoint allocation, gadget registration, and host-controller lifecycle reconstruction remain explicit unsupported bindings. Other subsystems, including I2C and SPI, require additional callback and lifecycle models.

Dataflow precision. The flow-sensitive analysis retains branch predicates and merges common switch/if RMW patterns into conditional expressions. It builds explicit source-level basic blocks with predecessor/successor edges, dominance/post-dominance, joins, goto edges, backedges, and loop headers. Simple parameter-only early exits, bounded forward gotos, and canonical statically bounded `for` loops can be lowered; backward gotos and unproved loops remain explicit blockers. The analysis still does not maintain a general abstract-store fixpoint for arbitrary paths, recursion, or external framework state. SVF alias analysis is available as an opt-in mode (`auto` or `required`) because its external toolchain and analysis cost would otherwise weaken deterministic reproduction. Multi-source manifests use linked bitcode and one WPA pass rather than per-translation-unit SVF; the default evaluation uses alias mode `off`.

Linux backend completeness. The deterministic Linux backend emits Kbuild-compilable output for 19/19 drivers and implements concrete platform, PCI, GPIO/IRQ, clock, power-management, and edu lifecycles. Compilation is not treated as semantic completion: only 7/19 is strictly ready. Common GPIO, clock, virtio, USB controller, and endpoint private fields are represented in the generated device state. GPIO IRQ initialization and clock-provider callbacks now preserve their concrete table bindings; clock rate callbacks retain source scalar arithmetic only after conservative private-state rebinding. Sodaville's source-derived GPIO/IRQ lifecycle is covered by structural assertions and Kbuild, but no real Sodaville hardware or equivalent device model was available. Correct USB callback signatures and ops-table entries are emitted, but incomplete USB core lifecycle bindings and other subsystem-specific objects are normalized only with explicit unsupported markers. Unknown transforms and unsafe computed addresses likewise block strict readiness.

Computed addresses. REHARNESSpreserves the complete runtime offset expression for indexed register banks and emits it directly when every term is lowerable. Such an access still lacks one static symbolic register name, but it is not inherently unsafe. Expressions containing unknown calls, unbound members, or `Top` remain explicit readiness blockers.

LLM reproducibility. LLM-assisted synthesis remains useful for candidates with sufficient structured context (13/19 in our readiness metric), but model versions, endpoints, and sampling introduce nondeterminism. For that reason, all headline extraction, compilation, and QEMU results use only deterministic stages; LLM outcomes are not included in the runtime claims.

Time complexity. The libclang parsing step is the dominant cost: parsing a typical Linux driver with full kernel headers takes 10–30 seconds. This is acceptable for batch analysis of individual drivers but would be prohibitive for whole-kernel analysis.

10.2 Future Work

Additional backends. The backend-binding architecture is designed for extensibility. We plan to add backends for RTOS targets (FreeRTOS, Zephyr) and verification targets (CBMC, SeaHorn) that consume the same `DeviceSpec` with different `BindSpec` files.

Stateful RIS modeling. The current RIScaptures op sequences but not device state transitions. A stateful extension that models register state machines (e.g., finite-state automata over register values) would enable stronger verification, such as checking that the driver never reads a register before it is initialized.

Whole-kernel analysis. Scaling extraction to thousands of kernel drivers requires incremental parsing, caching of macro tables and call graphs, and parallel execution. The modular nature of REHARNESS(per-driver, per-function extraction) makes this feasible.

Integration with safe-language translation. A natural next step is integrating REHARNESSwith C2Rust-style translators: the extracted RISand `DeviceSpec` can inform the type selection and ownership annotation decisions, particularly for MMIO pointers and interrupt handler contexts.

11 CONCLUSION

We presented REHARNESS, a pipeline for extracting formal register interaction specifications from C device drivers using libclang AST analysis with flow-sensitive dataflow and taint tracking. REHARNESSovercomes the four fundamental limitations of regex-based extraction: macro-defined register offsets are resolved via preprocessing records, wrapper functions are inlined through inter-procedural call-graph analysis, branch conditions are recorded through predicate stacking, and arithmetic address expressions are evaluated through symbolic abstract interpretation.

Beyond raw extraction, REHARNESSinfers backend-independent function and device semantics, composes them into a multi-backend code generation framework, and provides an optional closed-loop LLM-assisted synthesis module for complex targets. Across 19 drivers, it extracts 469 operations—356 symbolic, 74 fixed, and 25 computed-address—including 88 RMW patterns and 117 conditions, with a mean RIS quality score of 0.907. All three deterministic backends compile for 19/19 drivers, while the explicit 7/19 strict Linux readiness result prevents compilation from being mistaken for semantic completeness. Value-level and trace-level QEMU oracles pass for the deterministic edu and gpio-ftgpio010 outputs.

REHARNESSenables formal reasoning about driver behavior at the register level, a capability that has been missing from the driver analysis toolchain. By producing specifications in a formal language rather than ad-hoc formats, it provides a foundation for verification, testing, safe-language translation, and AI-assisted driver synthesis.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their constructive feedback.

REFERENCES

[1] A. Chou, J. Yang, B. Chelf, S. Hallem, and D. Engler. An empirical study of operating systems errors. In *Proc. SOSP*, 2001.
[2] M. M. Swift, B. N. Bershad, and H. M. Levy. Improving the reliability of commodity operating systems. In *Proc. SOSP*, 2003.
[3] T. Ball, E. Bounimova, B. Cook, V. Levin, J. Lichtenberg, C. McGarvey, B. Ondrusek, S. K. Rajamani, and A. Ustuner. Thorough static analysis of device drivers. In *Proc. EuroSys*, 2006.
[4] L. Ryzhyk, P. Chubb, I. Kuz, and G. Heiser. Dingo: Taming device drivers. In *Proc. EuroSys*, 2009.
[5] A. Kadav, M. J. Renzelmann, and M. M. Swift. Tolerating hardware device failures in software. In *Proc. SOSP*, 2009.
[6] T. Witkowski, N. Blanc, D. Kroening, and G. Weissenbacher. Model checking concurrent Linux device drivers. In *Proc. ASE*, 2007.
[7] C. Cadar, D. Dunbar, and D. Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proc. OSDI*, 2008.
[8] M. Emrath, P. V. D. Voort, and F. V. D. Lijn. C2Rust: Translating C to Rust. *https://c2rust.com/*, 2021.
[9] J. Roelofs, A. Kadav, and A. Cidon. Large language models as driver synthesizers. In *Proc. HotOS*, 2023.
[10] driver-harness: Regex-based driver register extraction. *https://github.com/...*, 2025.
[11] CRUST-Bench: A benchmark suite for C-to-Rust translation. 2024.
[12] L. Ryzhyk, P. Chubb, I. Kuz, E. L. Sueur, and G. Heiser. Automatic device driver synthesis with Termite. In *Proc. SOSP*, 2009.
[13] L. Ryzhyk, A. Walker, J. Keys, A. Legg, A. Raghunath, M. Stumm, and M. Vij. User-guided device driver synthesis. In *Proc. OSDI*, 2014.
[14] V. Chipounov, V. Kuznetsov, and G. Candea. S2E: A platform for in-vivo multi-path analysis of software systems. In *Proc. ASPLOS*, 2011.
[15] A. Baumann, P. Barham, P.-E. Dagand, T. Harris, R. Isaacs, S. Peter, T. Roscoe, A. Schüpbach, and A. Singhania. The multikernel: A new OS architecture for scalable multicore systems. In *Proc. SOSP*, 2009.
[16] Haiku operating system. *https://www.haiku-os.org/*.
[17] &inator: Correct, precise C-to-Rust interface translation. In *Proc. PLDI*, 2026.